

Cloud Computing

Exercise session 1

- Cloud Computing APL Introduction

Prepared by Wenfei Huang

15-Apr-26

Content

Exercise sessions, project and grading

Brief Kubernetes tutorial

Project introduction

Exercise Sessions

Exercise sessions

- Biweekly
- 5 exercise sheets, each 3-4 questions
- Tutorials
- Paper discussions

No Submission

- No grade
- Attendance & Attention
- The tasks help to understand the lecture
- ...are related to the written exam 😊

Project and Grading

Project

- It is a practical project related to cloud computing (Elasticity, Communication, ...)
- In groups

Grading

- 20% for aPL; 80% for Written exam
- aPL Grade = **if** (3/4 Quizzes passed) **then**
 Project grade
 else 5.0 (“Fail”)
- Final Grade = **if** (Exam passed) **then**
 Exam grade + aPL grade
 else 5.0 (“Fail”)

APL registration: 4 days (= 1st May) ahead of the first Quiz (= 5th May)

No registered aPL → No grade for aPL → No grade for the full module

Kubernetes Tutorial

Content

Kubernetes Architecture

Pod

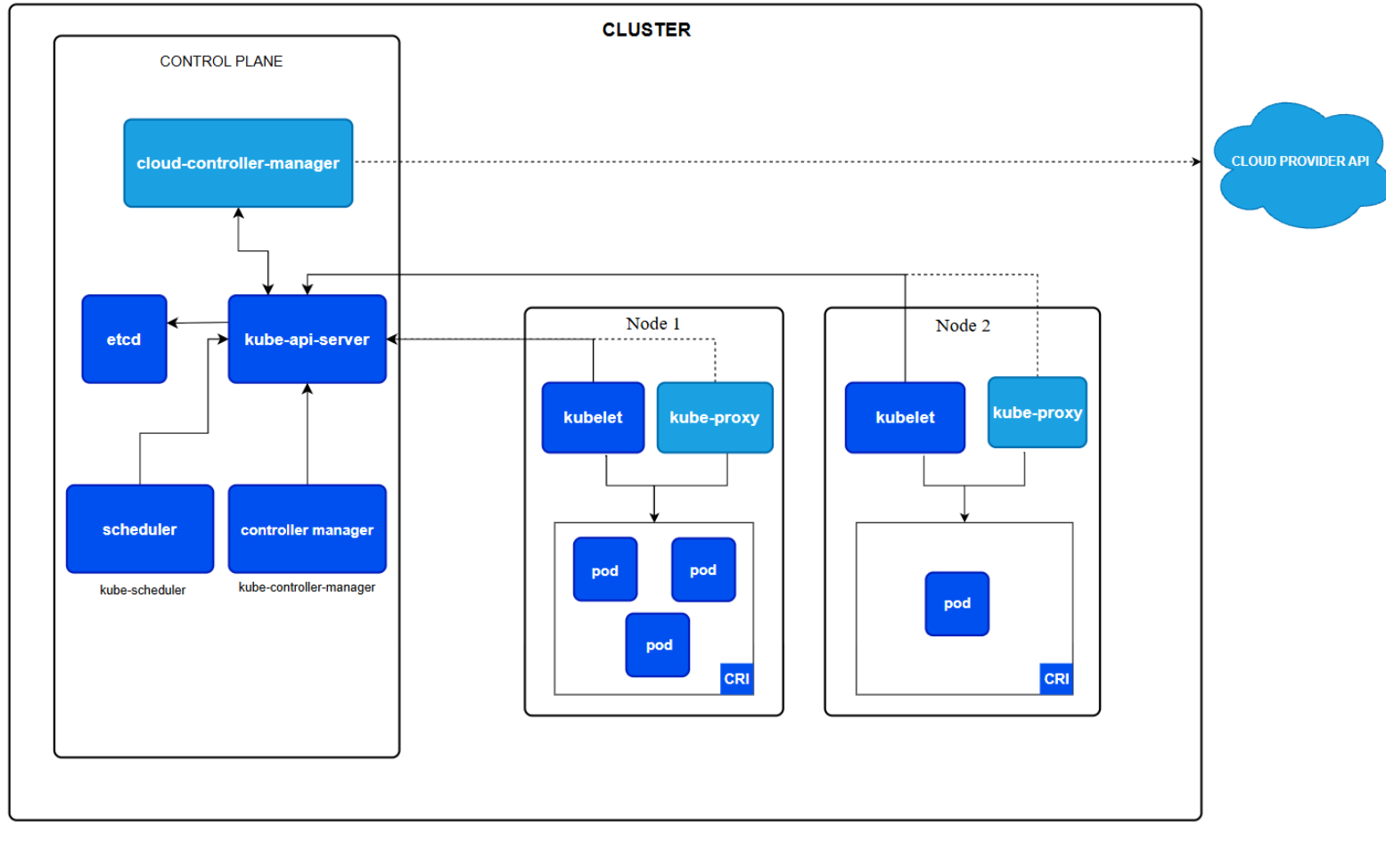
Deployment

Service

Horizontal Pod Autoscaler

Prometheus Monitoring Tool

Kubernetes (K8s) Architecture



kube-api-server: exposes the K8s http api

kube-scheduler: assigns Pod → node

kubelet: ensures Pods and their containers are running

CRI: container runtime interface

Pod: ...next...

<https://kubernetes.io/docs/concepts/architecture/>

Pod

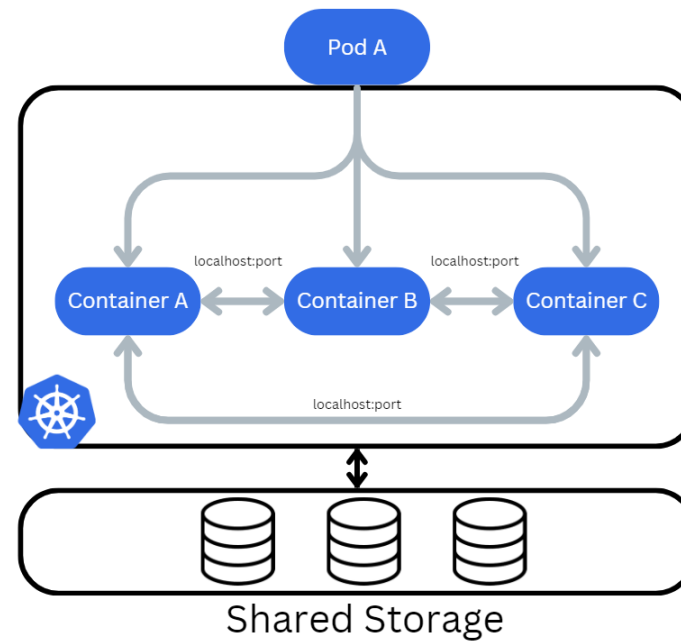
Smallest deployable unit in Kubernetes

Can have more than one Container

Pods can be deployed in two ways:

- Declarative (Using yaml files): *kubectl apply -f nginx_pod.yaml*
- Imperative: *kubectl run nginx --image=nginx:latest --port=80*

Use Declarative for the project



<https://www.couchbase.com/blog/pod-vs-container/>

nginx_pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: webserver
    image: nginx:latest
    ports:
    - containerPort: 80
```


Deployment

Wraps the Pods

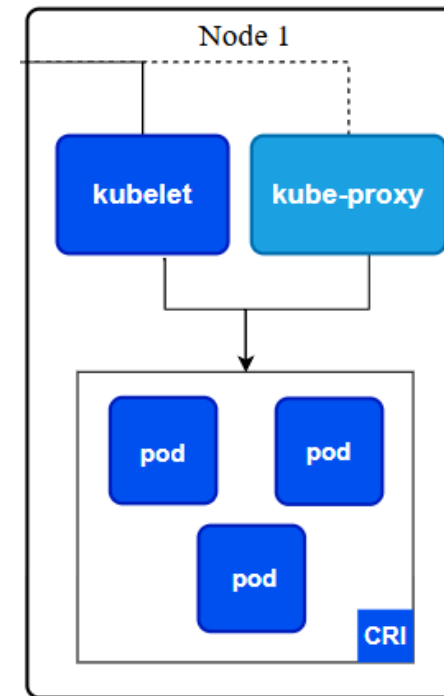
Manages the number of replicas

Provides self-healing

- If a Pod replica stops working, spawns a new pod to meet the number of replicas

Rollout process

- Update the application without down time



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Service

The way to expose your applications in Kubernetes

You do not need to know the IP address of individual Pods

Provides load balancing

Different service types

- ClusterIP: Service accessible within the cluster
- NodePort: Service accessible to outside from every cluster node

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - name: http
      protocol: TCP
      port: 8080
      targetPort: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 8080
      targetPort: 80
      nodePort: 30007
```

Horizontal Pod Autoscaler (HPA)

Autoscaler provided by Kubernetes

Works based on CPU utilization

By default, checks the CPU utilization every 15 seconds

Scale down stabilization

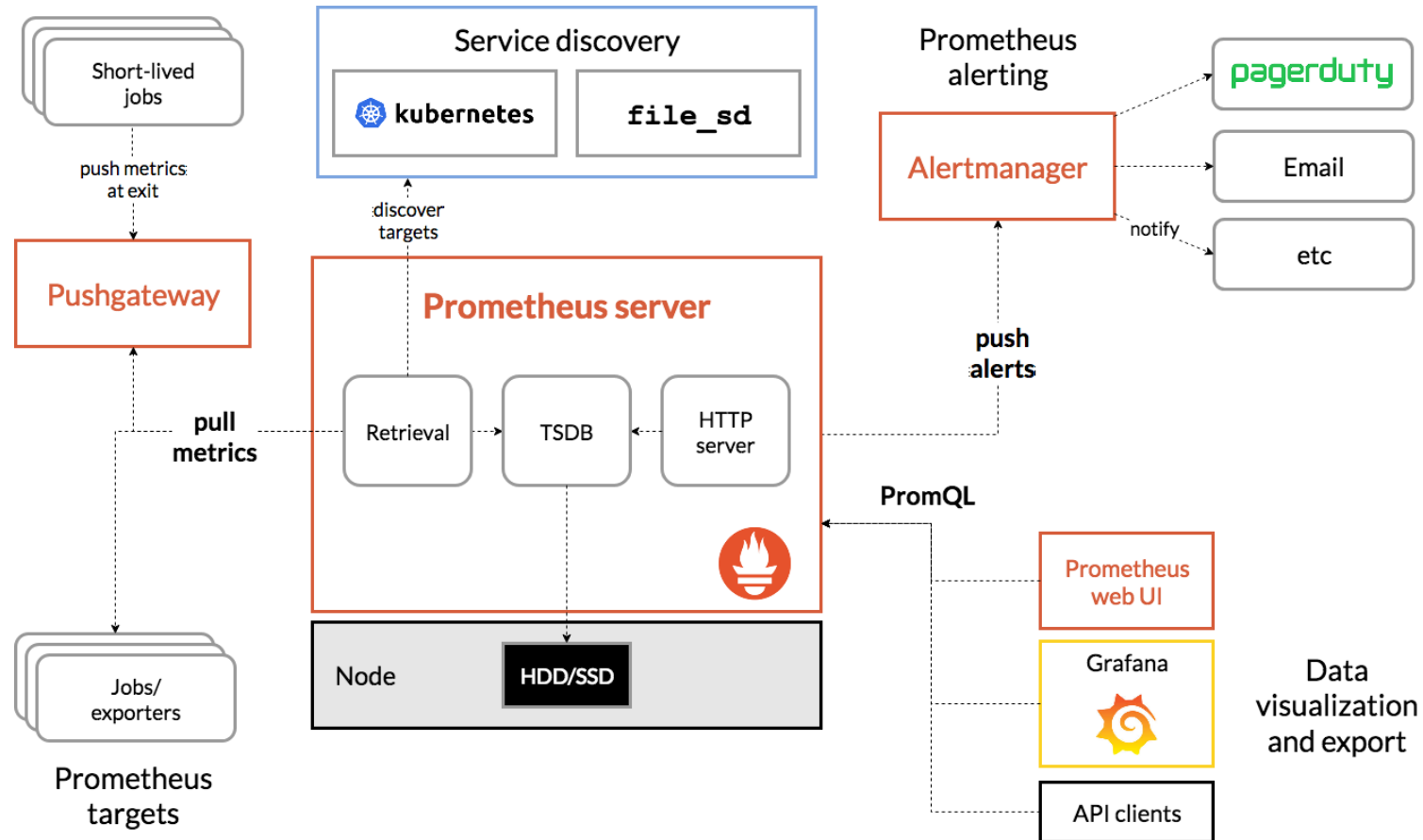
- 5 minutes by default

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80
```

*$desiredReplicas = \text{ceil}[currentReplicas * (currentMetricValue / desiredMetricValue)]$*

<https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

Prometheus Monitoring Tool



<https://prometheus.io/docs/introduction/overview/>

Project Introduction: Elastic ML Inference Serving

Content

Teaching Objectives

Application Introduction

Your Project

System Design – Components

Timeline

Teaching Objectives

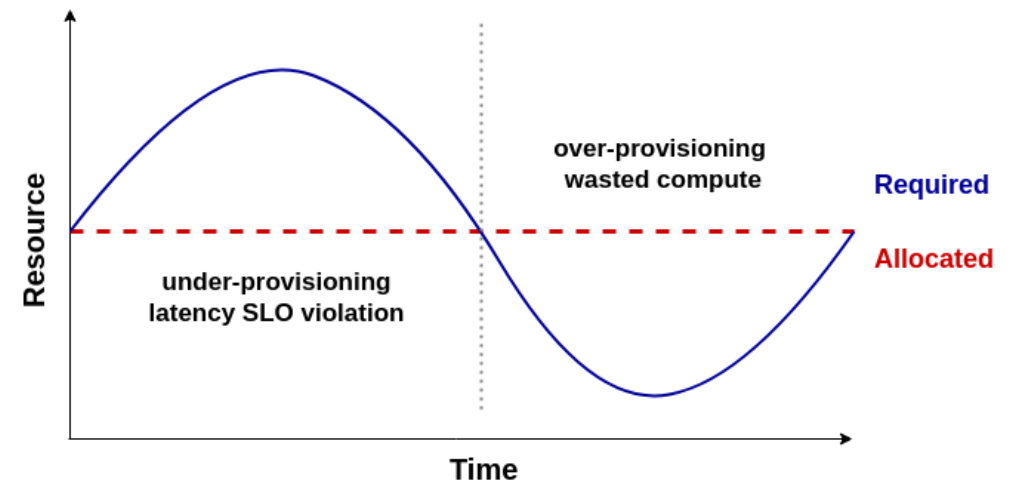
Understanding how to achieve elasticity for services in the Cloud

- How to react when the service is under/over utilized?

Getting familiar with CaaS and its cost model

Hands-on with a Cloud Container orchestration tool

Understanding the concept of Service Level Objective



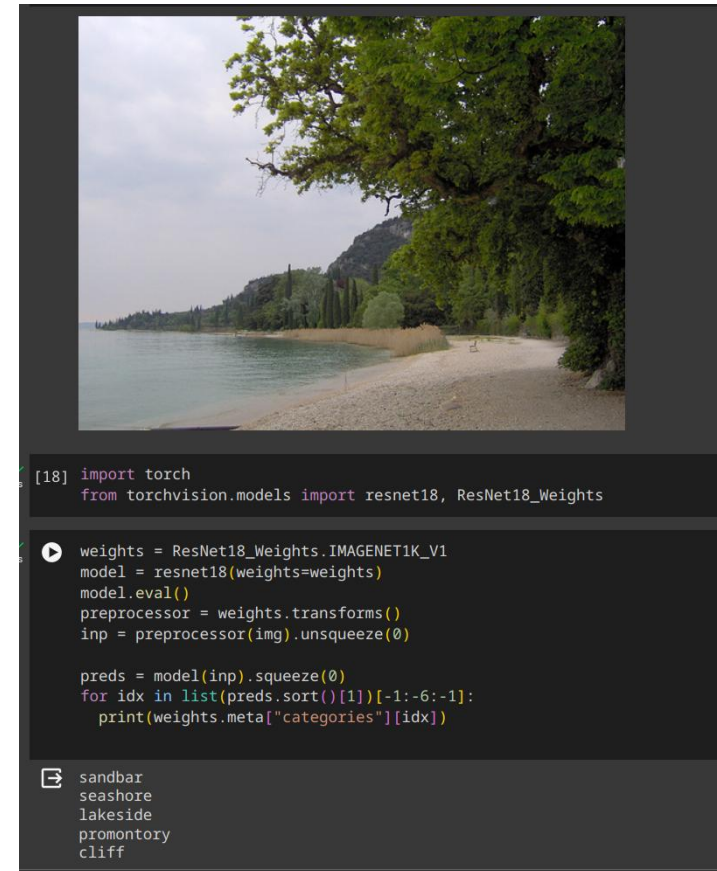
Application Introduction

ML Inference: Using pre-trained ML model to predict online unseen data

- ML Inference services are resource intensive
- ML Inference services are latency sensitive

In this project we focus on ML image classification

- Users send images as queries (inputs)
- The ML service returns labels (outputs)
- You will use ResNet18 ML model for the classification



Your Project

Implementing autoscaling for image classification Inference service in a K8s cluster

- More details on the system design and components are in the following slides
- Make sure every component works correctly
- The models must be executed on CPU and not GPU
- Be creative in designing and implementing the Autoscaler component!

Achieve server-side latency of $< 0.5s$ for the queries

Your autoscaler should outperform Kubernetes HPA

- Run the final experiment once using your autoscaler and twice using HPA (70% and 90% CPU targets) to:
 - Compare the service 99th percentile latency from the start to the end of the experiment
 - Compare the number of CPU cores from the start to the end of the experiment
 - Plot a figure for the metrics time-series comparison (x-axis being time and y-axis being the metrics)

You run the cluster and the experiments on your laptops using Minikube

System Design - Components

Here you see some components you must implement for your project:

Load Tester

Dispatcher

Containerized Image Classification Service

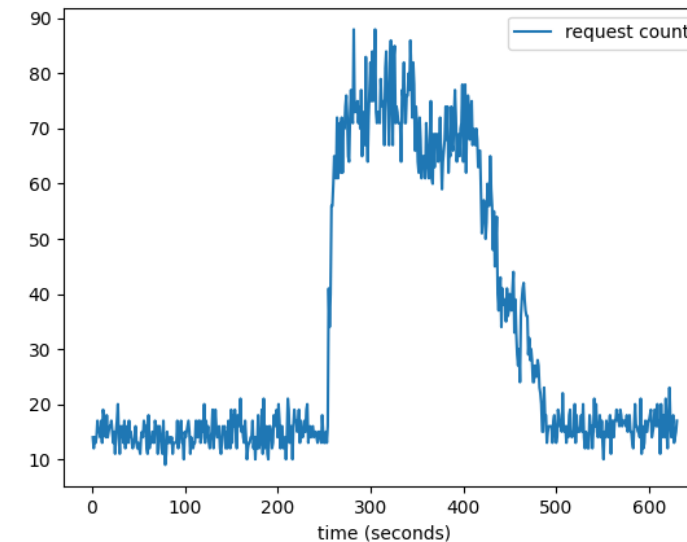
Monitoring Tool

Autoscaler

System Design – Components – Load Tester

Load Tester

Sending queries to the ML service based on a workload pattern (queries/s). The workload pattern will be given to you.



Adapted from: <https://archive.org/details/archiveteam-twitter-stream-2021-08>

For query images, you can use the samples at:

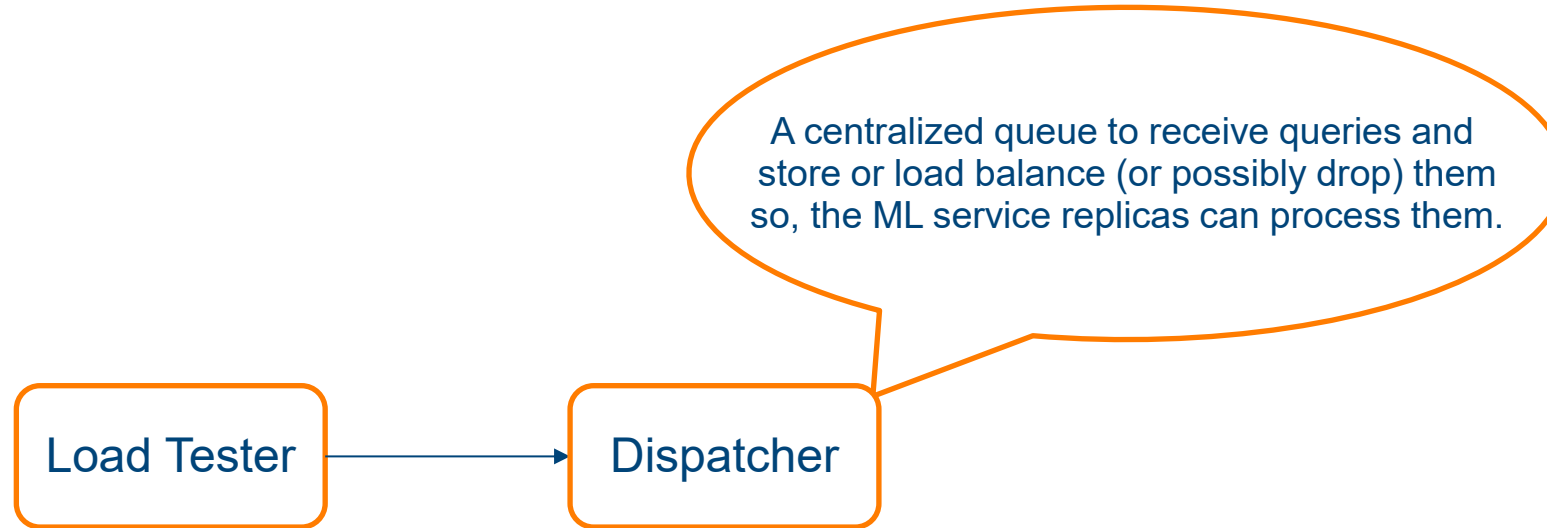
<https://github.com/EliSchwartz/imagenet-sample-images>

For load tester, you can use:

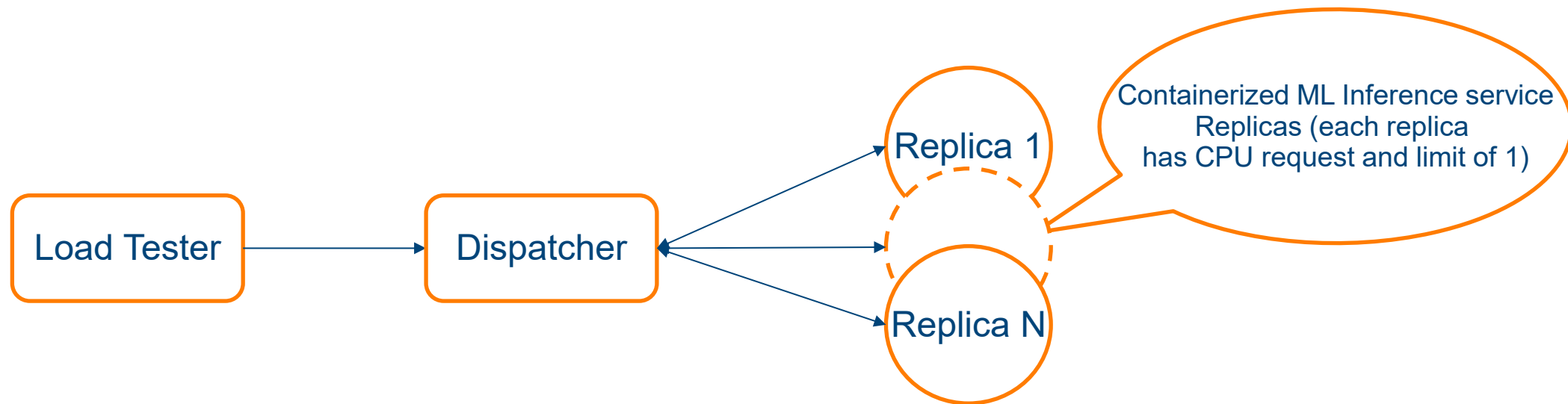
https://github.com/reconfigurable-ml-pipeline/load_tester

Installation: `pip install git+https://github.com/reconfigurable-ml-pipeline/load_tester`

System Design – Components – Dispatcher

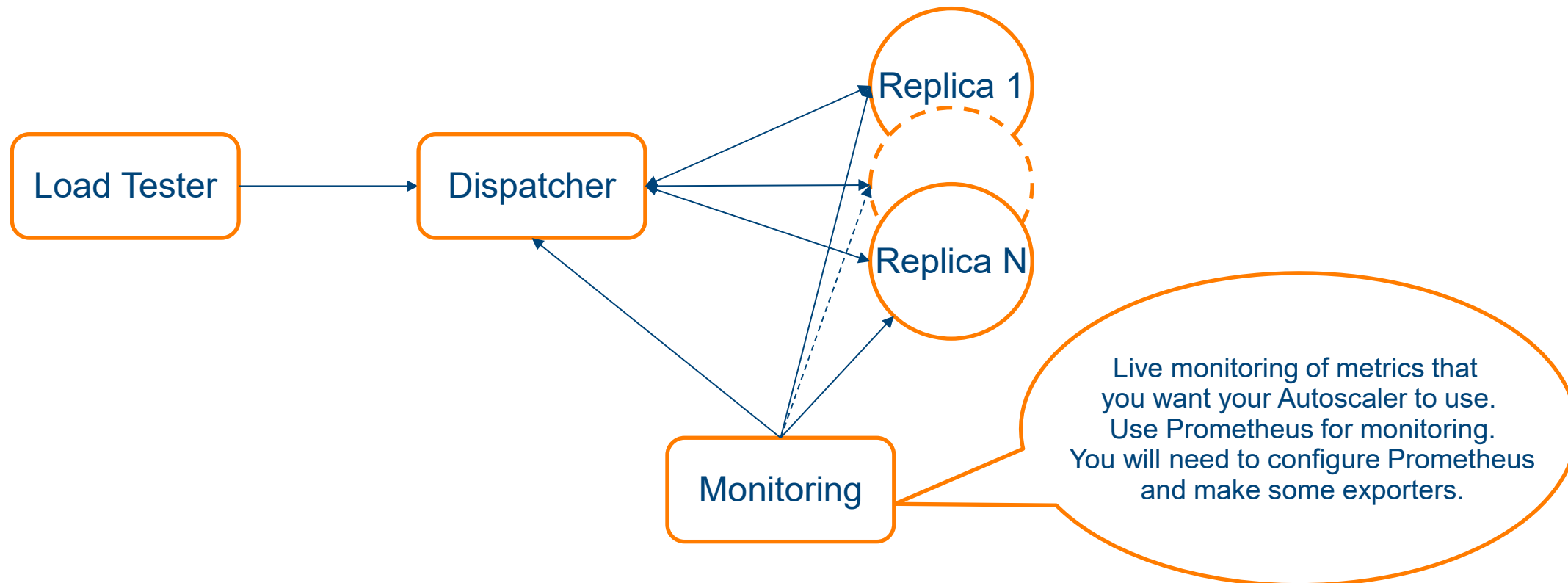


System Design – Components – ML Inference Service

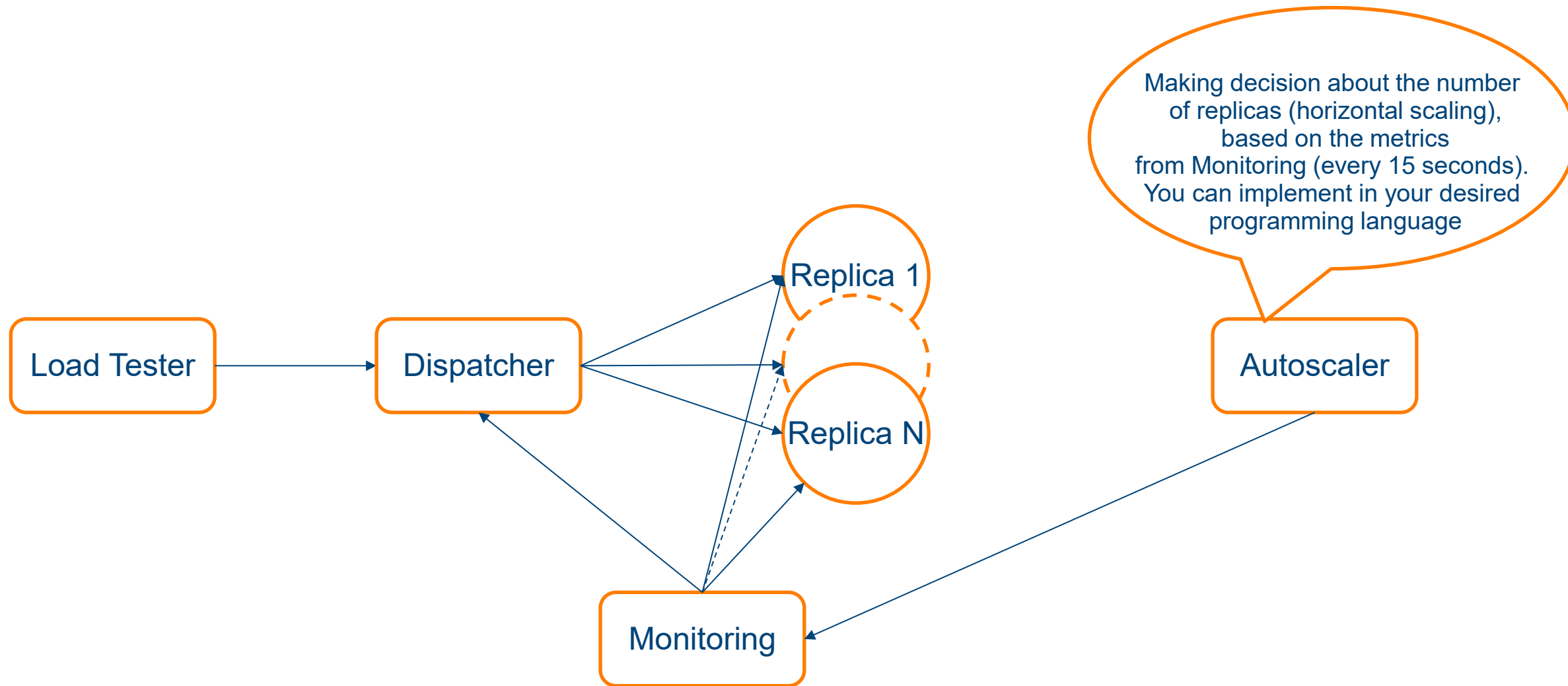


The queuing only happens at the Dispatcher. Replicas do not queue any query, they just perform inference on a query at a time. Use a simple Python server like Flask or FastAPI, or use systems like TorchServe or TensorFlow Serving to serve the Resnet18 ML model.

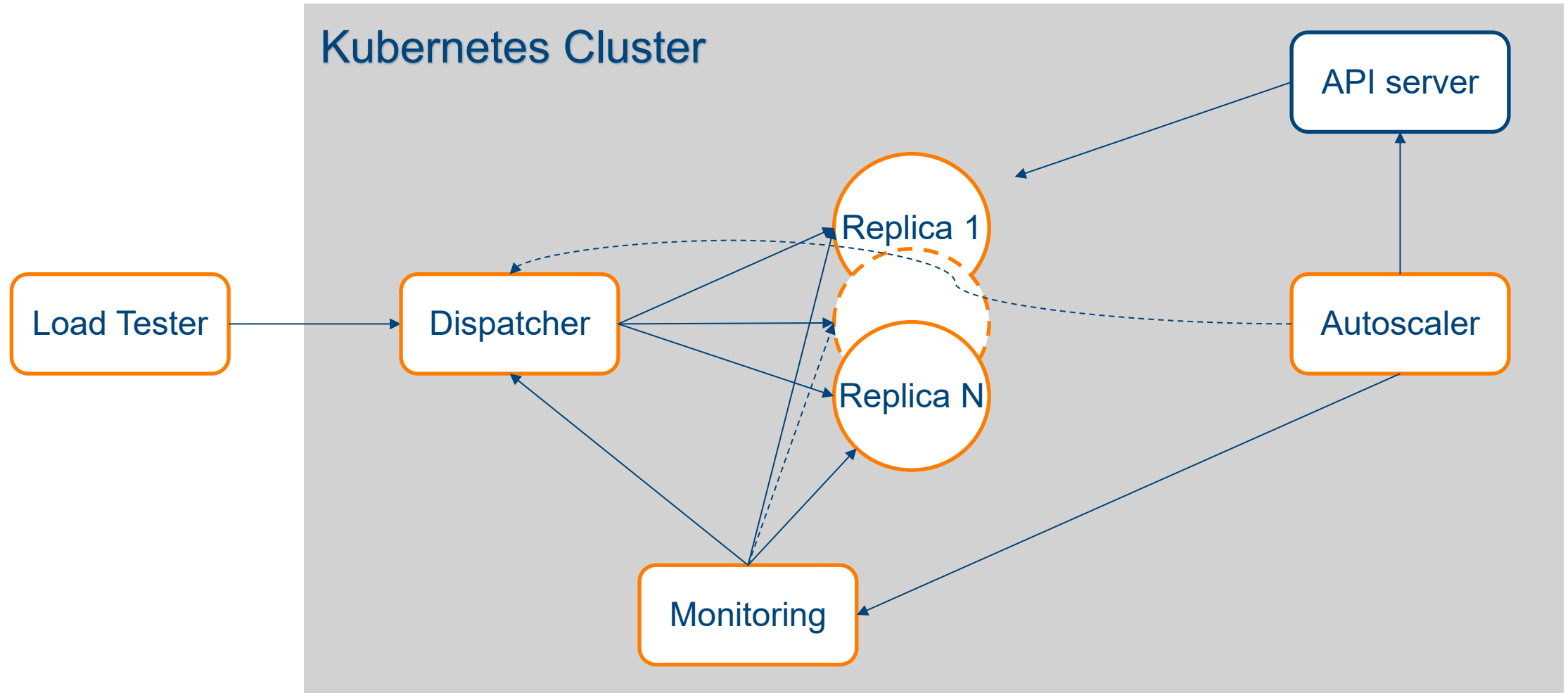
System Design – Components – Monitoring



System Design – Components – Autoscaler



System Design



Timeline

Project Submission: 25th June 2025

Presentations: 30th June - 07th July 2025

Course Moodle

**Moodle Subscription Key:
CloudC!26**

Questions



Wenfei Huang

Technische Universität Ilmenau

Department of Computer Science and Automation
Distributed Systems and Operating Systems Group